



IT GENIUS
THINK IT • BE GENIUS

Python Basic 2026

ปูพื้นฐานการเขียนโปรแกรมภาษา Python
ตั้งแต่เริ่มต้นจนถึงการประยุกต์ใช้งานจริง

Beginner Course • Hands-on Workshop • 2 Days (12 Hours)

Python 3.13/3.14 • uv • VS Code • PyCharm



พื้นฐานแน่น
เข้าใจง่าย



ลงมือทำจริง
เรียนรู้จากการปฏิบัติ



ต่อยอดได้จริง
ใช้งานได้ทันที

```
main.py
1 # เริ่มต้นกับ Python
2 name = "IT Genius"
3 message = f"Hello, {name}!"
4 print(message)
5
6 for i in range(1, 6):
7     print(f"Count: {i}")
```

```
$ uv init my-project
$ cd my-project
$ uv run main.py
Hello, Python!
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```



เรียนจบ
นำไปใช้งานจริง
ได้ทันที!



IT GENIUS
THINK IT • BE GENIUS

Python Basic 2026

ปูพื้นฐานการเขียนโปรแกรมภาษา Python
ตั้งแต่เริ่มต้นจนถึงการประยุกต์ใช้งานจริง

Beginner Course • Hands-on Workshop • 2 Days (12 Hours)

Python 3.13/3.14 • uv • VS Code • PyCharm



พื้นฐานแน่น
เข้าใจง่าย



ลงมือทำจริง
เรียนรู้จากการปฏิบัติ



ต่อยอดได้จริง
ใช้งานได้ทันที

```
main.py
1 # เริ่มต้นกับ Python
2 name = "IT Genius"
3 message = f"Hello, {name}!"
4 print(message)
5
6 for i in range(1, 6):
7     print(f"Count: {i}")
```

```
$ uv init my-project
$ cd my-project
$ uv run main.py
Hello, Python!
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```

Files

- project/
- main.py
- data.csv
- README.md

DAY 2



เรียนจบ
นำไปใช้งานจริง
ได้ทันที!

จากโปรแกรมในหน่วยความจำ สู่ระบบที่เก็บข้อมูลจริง

DAY 1 — เสร็จแล้ว

ข้อมูลฝังในโค้ด `sample_inventory()`

ข้อมูลเป็น dict

ปิดโปรแกรม = ข้อมูลหาย

หน้าจอเป็นข้อความ

ผิดพลาดแล้วโปรแกรมพัง



DAY 2 — วันนี้

ข้อมูลอยู่ในไฟล์ CSV/JSON และ PostgreSQL

ข้อมูลเป็น class Product

ปิดแล้วข้อมูลยังอยู่

หน้าจอเป็น GUI จริง

จับ error ได้อย่างสง่างาม

เมื่อวานเราออกแบบให้ `main.py` มีแต่ส่วนติดต่อผู้ใช้ ส่วนตรรกะอยู่ใน `src/pyshop` ทั้งหมด **วันนี้จะพิสูจน์ว่าการออกแบบแบบนั้นคุ้มค่า** เพราะเราจะเปลี่ยนแหล่งข้อมูลและเปลี่ยนหน้าจอ โดยแทบไม่แตะโค้ดตรรกะเลย

```
cd C:\PythonTraining\pyshop && uv run main.py
```


AGENDA

กำหนดการวันที่ 2

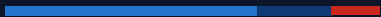
09:00-09:15	ทบทวนวันที่ 1 และตรวจสอบความพร้อมโปรเจกต์
09:15-10:30	Module 7 การจัดการแฟ้มข้อมูล (Text / CSV / JSON / pathlib)
10:30-12:00	Module 8 การเขียนโปรแกรมเชิงวัตถุ (OOP)
12:00-13:00	พักรกลางวัน
13:00-13:45	Module 9 การจัดการข้อผิดพลาดและการ Debug
13:45-14:45	Module 10 การเชื่อมต่อฐานข้อมูล PostgreSQL
14:45-15:15	Module 11 Git Version Control
15:15-15:45	Module 12 Python GUI ด้วย Tkinter / CustomTkinter

07

MODULE 07 · 09:15-10:30

การจัดการแฟ้มข้อมูล

CSV กับ JSON คือสองรูปแบบที่ผ่านจะเจอบ่อยที่สุดในชีวิตการทำงาน



คำสั่ง with และโหมดการเปิดไฟล์

```
# วิธีเดิม: เกิด error ไฟล์จะไม่ถูกปิด
f = open("products.txt", "r",
        encoding="utf-8")
content = f.read()
f.close()
```

```
# วิธีที่ถูกต้อง: with ปิดไฟล์ให้เองเสมอ
with open("products.txt", "r",
        encoding="utf-8") as f:
    content = f.read()
# ออกจากบล็อก = ไฟล์ถูกปิดแล้วแน่นอน
```

โหมด	ความหมาย	ถ้าไฟล์ไม่มี	ถ้าไฟล์มีอยู่แล้ว
"r"	อ่านอย่างเดียว	FileNotFoundError	อ่านได้
"w"	เขียนทับทั้งไฟล์	สร้างใหม่	ล้างข้อมูลเดิมทิ้งทั้งหมด
"a"	เขียนต่อท้าย	สร้างใหม่	เขียนต่อท้ายข้อมูลเดิม

`encoding="utf-8"` สำคัญมากสำหรับภาษาไทย บน Windows คำเริ่มต้นไม่ใช่ UTF-8 ถ้าไม่ระบุ ข้อความไทยจะกลายเป็นตัวประหลาดหรือเกิด `UnicodeDecodeError`

อ่าน เขียน และต่อท้ายไฟล์ Text

การอ่าน 3 วิธี

```
with open(p, encoding="utf-8") as f:
    content = f.read()      # ทั้งไฟล์ก้อนเดียว

with open(p, encoding="utf-8") as f:
    lines = f.readlines()   # เป็น list

# แนะนำที่สุด: อ่านทีละบรรทัด
with open(p, encoding="utf-8") as f:
    for no, line in enumerate(f, start=1):
        print(f"{no}: {line.rstrip()}")
```

ถ้าไฟล์ขนาด 2 GB สองวิธีแรกจะโหลดทั้งหมดเข้าหน่วยความจำจนเครื่องค้าง แต่วิธีที่สามใช้หน่วยความจำเท่าเดิมเสมอ

การเขียนและต่อท้าย

```
# เขียนทับทั้งไฟล์
with open(p, "w", encoding="utf-8") as f:
    f.write("รายการสินค้า\n")
    for product in products:
        f.write(f"- {product}\n")

# ต่อท้ายไฟล์เดิม (Log)
with open("activity.log", "a",
        encoding="utf-8") as f:
    stamp = datetime.now().strftime(
        "%Y-%m-%d %H:%M:%S")
    f.write(f"[{stamp}] เพิ่มสินค้า\n")
```

โหมด "w" คือขาดกรเขียน เปิดไฟล์แล้วแม้ยังไม่ได้เขียนอะไร ข้อมูลเดิมก็หายหมดแล้ว ถ้าตั้งใจจะเพิ่มข้อมูลให้ใช้ "a" เสมอ

จัดการ Path ของไฟล์ด้วย pathlib

```
from pathlib import Path

data_dir = Path("data")
csv_path = data_dir / "products.csv"

csv_path.name      # products.csv
csv_path.stem      # products
csv_path.suffix     # .csv
csv_path.parent    # data
csv_path.exists()
csv_path.is_file()
```

```
data_dir.mkdir(exist_ok=True)
Path("data/backup/2026").mkdir(
    parents=True, exist_ok=True)
```

```
csv_path.write_text(text, encoding="utf-8")
content = csv_path.read_text(encoding="utf-8")

for file in data_dir.glob("*.csv"):
    print(file.name)

for file in Path(".").rglob("*.py"):
    print(file)

csv_path.unlink(missing_ok=True)
```

เคล็ดลับสำคัญ หาโฟลเดอร์ของสคริปต์เอง เพื่อให้หาไฟล์เจอแม้ถูกรันจากโฟลเดอร์อื่น

```
BASE_DIR = Path(__file__).resolve().parent
DATA_FILE = BASE_DIR / "data" / "products.csv"
```

วิธีเก่าใช้การต่อข้อความ เช่น "data" + "\\\" + "products.csv" ซึ่งพบบน macOS และ Linux ส่วน pathlib ใช้ / เชื่อม path ได้เลยและทำงานถูกต้องทุกระบบปฏิบัติการ

อ่านและเขียนไฟล์ CSV

อ่านด้วย csv.DictReader

```
import csv

with open(CSV_PATH, "r",
          encoding="utf-8-sig",
          newline="") as f:
    for row in csv.DictReader(f):
        products.append({
            "code": row["code"],
            "name": row["name"],
            "price": float(row["price"]),
            "stock": int(row["stock"]),
        })
```

utf-8-sig ไฟล์ CSV จาก Excel บน Windows มักมีอักขระซ่อนชื่อ BOM ต้นไฟล์ ทำให้ชื่อคอลัมน์แรกเพี้ยน utf-8-sig จะตัด BOM ออกให้อัตโนมัติ

เขียนด้วย csv.DictWriter

```
FIELDNAMES = ["code", "name", "price",
               "stock", "category"]

with open(OUT_PATH, "w",
          encoding="utf-8-sig",
          newline="") as f:
    writer = csv.DictWriter(
        f, fieldnames=FIELDNAMES)
    writer.writeheader()
    writer.writerows(products)
```

newline="" ป้องกันบรรทัดว่างแทรกระหว่างแถวบน Windows ซึ่งเป็นข้อกำหนดของโมดูล csv โดยตรง

อ่านและเขียนไฟล์ JSON

```
import json

# เขียน
with open(p, "w", encoding="utf-8") as f:
    json.dump(products, f,
               ensure_ascii=False, indent=2)

# อ่าน
with open(p, "r", encoding="utf-8") as f:
    loaded = json.load(f)

# datetime แปลงไม่ได้ ต้องแปลงเป็น str ก่อน
json.dumps({"time":
            datetime.now().isoformat()})
```

ensure_ascii=False สำคัญมากสำหรับภาษาไทย ถ้าไม่ใส่ ข้อความไทยจะถูกเข้ารหัสเป็น \u0e40\u0e21 ซึ่งอ่านกลับได้แต่มนุษย์อ่านไม่ออก

Python	JSON
dict	object { }
list, tuple	array []
str	string
int, float	number
True / False	true / false
None	null
datetime	แปลงไม่ได้

CSV กับ JSON เลือกใช้อันไหน

ประเด็น	CSV	JSON
โครงสร้าง	ตารางแบบ แถว x คอลัมน์	ซ้อนกันได้หลายชั้น
เปิดด้วย Excel	ได้ทันที	ไม่ได้
ขนาดไฟล์	เล็กกว่า	ใหญ่กว่า มีชื่อฟิลด์ซ้ำทุกแถว

```
# สำรองก่อนเขียนทับเสมอ
if path.exists():
    backup = path.with_name(
        f"{path.stem}_backup{path.suffix}")
    shutil.copy2(path, backup)

path.unlink(missing_ok=True)
```

shutil.rmtree() ลบทุกอย่างในโฟลเดอร์โดยไม่ถามและไม่ไป Recycle Bin ก่อนใช้งานจริงให้พิมพ์ path ออกมาดูก่อนเสมอ

WORKSHOP 7.1

เปลี่ยน PyShop Mini ให้อ่าน-เขียนไฟล์จริง

ย้ายข้อมูลสินค้าจาก `sample_inventory()` ที่ฝังในโค้ด ไปอยู่ในไฟล์ CSV แล้วเพิ่มความสามารถส่งออกรายงานเป็น CSV และ JSON

```
# src/pyshop/storage.py
def ensure_data_dir() -> None
def load_products(path=CSV_PATH) -> list
def save_products(products, path) -> None
def export_json(products, path) -> None
def write_log(message: str) -> None

# main.py
inventory = load_products()

case "7": save_products(inventory)
case "8": export_json(inventory)
```

ผลลัพธ์ที่คาดหวัง

เพิ่มสินค้าใหม่ผ่านเมนู 3 แล้วเลือกเมนู 7 เพื่อบันทึก จากนั้น ปิดโปรแกรมแล้วเปิดใหม่ สินค้าที่เพิ่มต้องยังอยู่ และในโฟลเดอร์ data ต้องมีไฟล์สำรอง .bak.csv เกิดขึ้น

โจทย์ต่อยอด

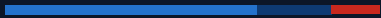
เขียนฟังก์ชัน `import_csv(path)` ที่นำเข้าไฟล์ CSV จากภายนอกแล้ว รวมกับข้อมูลเดิม โดยถ้ารหัสซ้ำให้ปรับจำนวนแทนการเพิ่มซ้ำ

08

MODULE 08 · 10:30-12:00

การเขียนโปรแกรมเชิงวัตถุ (OOP)

OOP คือการมัดข้อมูลกับพฤติกรรมที่เกี่ยวข้องไว้ด้วยกัน ทำให้ข้อมูลดูแลตัวเองได้



แนวคิดการสร้างโปรแกรมเชิงวัตถุ

```
product = {  
    "name": "เมาส์",  
    "price": 890,  
}  
  
def stock_value(product)  
def restock(product, qty)
```



```
class Product:  
    name, price, stock    ← ข้อมูล  
  
    def stock_value()      ← พหุติกรรม  
    def restock(qty)       อยู่ด้วยกัน  
    def is_low_stock()
```

ข้อมูลกับฟังก์ชันแยกกันอยู่ พิมพ์ key ผิดไม่มีใครเตือน

มัดรวม แก่ค่าผ่านเมธอดที่ควบคุมได้ พิมพ์ชื่อผิด Editor เตือนทันที

Class พิมพ์เขียวหรือแม่แบบ เปรียบเหมือนแบบแปลนบ้าน

Object / Instance สิ่งที่สร้างจาก Class คือ บ้านหลังจริง

Attribute ข้อมูลของ Object เช่น สีบ้าน จำนวนห้อง

Method ฟังก์ชันของ Object เช่น เปิดประตู เปิดไฟ

Constructor เมธอด `__init__` ที่ทำงานตอนสร้าง Object

self ตัวแทนของ Object ตัวที่กำลังทำงานอยู่ แปลว่า "ตัวฉันเอง"

การสร้างคลาสและ Constructor

```
class Product:
    """คลาสแทนสินค้า 1 รายการ"""

    # Class Attribute – ใช้ร่วมกันทุก object
    VAT_RATE = 0.07
    LOW_STOCK_LIMIT = 10

    def __init__(
        self, code: str, name: str,
        price: float, stock: int = 0,
        category: str = "ทั่วไป",
    ) -> None:
        # Instance Attribute – ของตัวเอง
        self.code = code.strip().upper()
        self.name = name.strip()
        self.price = float(price)
        self.stock = int(stock)
        self.category = category
```

```
mouse = Product("PRD-001",
                "เมาส์ไร้สาย", 890, 25)
keyboard = Product("PRD-002",
                  "คีย์บอร์ด", 1290, 8)

mouse.name          # เมาส์ไร้สาย
keyboard.stock      # 8
Product.VAT_RATE    # 0.07
```

เมื่อเรียก `mouse.some_method()` Python จะส่ง `mouse` เข้าไปเป็นพารามิเตอร์แรกโดยอัตโนมัติ ซึ่งรับด้วยชื่อ `self` ตามธรรมเนียม **ทุกเมธอดของ instance ต้องมี `self` เป็นพารามิเตอร์แรกเสมอ**

Method — พฤติกรรมของวัตถุ

```
# เมธอดคำนวณ
def stock_value(self) -> float:
    return self.price * self.stock

def is_low_stock(self) -> bool:
    return 0 < self.stock < self.LOW_STOCK_LIMIT

def status(self) -> str:
    if self.stock <= 0:
        return "สินค้าหมด"
    if self.is_low_stock():
        return "ใกล้หมด"
    return "ปกติ"

# เมธอดเปลี่ยนสถานะ
def sell(self, qty: int) -> float:
    if qty > self.stock:
        raise ValueError(
            f"คงเหลือเพียง {self.stock} ชิ้น")
    self.stock -= qty
    return self.price_with_vat() * qty

def __str__(self) -> str:
    return f"{self.code} {self.name}"
```

Magic Method	ทำงานเมื่อ
<code>__init__</code>	สร้าง object
<code>__str__</code>	<code>print(obj)</code> , <code>str(obj)</code>
<code>__repr__</code>	แสดงให้นักพัฒนาดู (ดีบั๊ก)
<code>__eq__</code>	<code>obj1 == obj2</code>
<code>__len__</code>	<code>len(obj)</code>
<code>__lt__</code>	<code>sorted(objects)</code>

การห่อหุ้มข้อมูล (Encapsulation)

```
class Product:
    def __init__(self, code, price, stock=0):
        self.code = code
        self._price = price      # ภายใน
        self.__stock = stock    # ซ่อนจริง

    @property
    def price(self) -> float:
        return self._price

    @price.setter
    def price(self, value: float) -> None:
        if value < 0:
            raise ValueError("ราคาต้องไม่ติดลบ")
        self._price = value

    @property
    def stock_value(self) -> float:
        return self._price * self.__stock
```

```
p.price          # 890   เรียกเหมือน attribute
p.stock_value    # คำนวณสด ไม่ต้องใส่วงเล็บ
p.price = 950    # เรียก setter ตรวจสอบค่าให้
p.price = -100   # ValueError
p.stock = 999    # AttributeError
p.restock(10)    # ต้องผ่านเมธอดที่ควบคุมไว้
```

รูปแบบ	ความหมาย
name	สาธารณะ ใช้ได้อิสระ
_name	ภายใน ไม่ควรใช้จากข้างนอก (ไม่บังคับ)
__name	ซ่อนด้วย Name Mangling

ต่างจาก Java ที่มี private บังคับจริง Python เชื่อในหลัก "We are all consenting adults here" คือใช้ธรรมเนียมเตือนกันแทนการบังคับด้วยภาษา

การสืบทอด (Inheritance) และการ Override

```
class Product:
    def shipping_fee(self) -> float:
        return 50.0

    def describe(self) -> str:
        return f"{self.name} {self.price:,.0f} บาท"

class DigitalProduct(Product):
    def __init__(self, code, name, price, url):
        super().__init__(code, name, price,
                        stock=999999)
        self.download_url = url

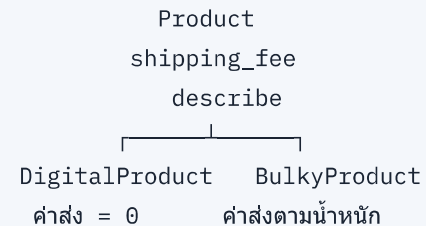
    def shipping_fee(self) -> float:
        return 0.0          # Override

    def describe(self) -> str:
        return f"{super().describe()} [ดาวน์โหลด]"

class BulkyProduct(Product):
    def shipping_fee(self) -> float:
        return 100.0 + self.weight_kg * 20
```

```
# Polymorphism
for item in items:
    print(item.describe(),
          item.shipping_fee())

isinstance(digital, Product)      # True
issubclass(DigitalProduct,
           Product)                # True
```



อย่าใช้ Inheritance พร่ำเพรื่อ ใช้เมื่อความสัมพันธ์เป็นแบบ **is-a** จริง ๆ ถ้าเป็นแบบ **has-a** เช่น Order มี Product ให้ใช้ Composition แทน

Dataclass — เขียนคลาสเก็บข้อมูลแบบกระชับ

```
from dataclasses import dataclass, field, asdict
```

```
@dataclass
class Product:
    code: str
    name: str
    price: float
    stock: int = 0
    category: str = "ทั่วไป"
    tags: list[str] = field(default_factory=list)
```

```
VAT_RATE = 0.07
```

```
def __post_init__(self) -> None:
    self.code = self.code.strip().upper()
    if self.price < 0:
        raise ValueError("ราคาต้องไม่ติดลบ")
```

```
@property
def stock_value(self) -> float:
    return self.price * self.stock
```

เขียนเอง

ใช้ @dataclass

เขียน __init__ เอง 10 บรรทัด

สร้างให้อัตโนมัติ

เขียน __repr__ เอง

ได้ฟรี พร้อมทุกฟิลด์

เขียน __eq__ เอง

ได้ฟรี เทียบทุกฟิลด์

แปลงเป็น dict ต้องทำเอง

asdict(obj)

ค่าเริ่มต้นที่เป็น list ต้องใช้ field(default_factory=list)
ห้ามใส่ = [] ตรง ๆ

@dataclass(frozen=True) ทำให้ object แกว้ค่าไม่ได้เลยหลังสร้าง
เหมาะกับข้อมูลที่ไม่ควรเปลี่ยน เช่น รายการในใบเสร็จที่ออกไปแล้ว

WORKSHOP 8.1

ออกแบบคลาสสำหรับ PyShop Mini

เปลี่ยนจาก dict เป็น class เพิ่มรูปแบบ พร้อมคลาส Inventory ที่ดูแลรายการสินค้าทั้งหมด

```
# src/pyshop/product.py
@dataclass
class Product:
    stock_value, price_with_vat (property)
    status(), restock(), sell()
    to_dict(), from_dict() (classmethod)

@dataclass
class Inventory:
    __len__(), __iter__()
    add(), remove(), find(), search()
    total_value, total_quantity
    low_stock(), by_category(), sorted_by()
```

สังเกตความต่างจากเมื่อวาน

`inv.find("PRD-001").sell(5)` อ่านแล้วเข้าใจทันทีว่าทำอะไร
ต่างจากเมื่อวานที่ต้องเขียน `product["stock"] = product["stock"] - 5` และไม่มีอะไรกันไม่ให้ขายเกินจำนวนที่มี

โจทย์ต่อยอด

สร้างคลาสลูก `DigitalProduct(Product)` ที่ไม่จำกัดสต็อกและไม่มีค่าจัดส่ง แล้วเพิ่มเข้า `Inventory` เดียวกัน สังเกตว่า `Inventory` ทำงานกับทั้งสองคลาสได้โดยไม่ต้องแก้ไขอะไรเลย

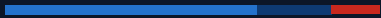
```
uv run -m src.pyshop.product
```

09

MODULE 09 · 13:00-13:45

การจัดการข้อผิดพลาดและการ Debug

บอกผู้ใช้งานว่าเกิดอะไรขึ้น และไม่ทำให้ข้อมูลเสียหาย



ประเภทของ Error และ try / except

Exception	ตัวอย่าง
ValueError	<code>int("abc")</code>
TypeError	<code>"5" + 5</code>
KeyError	<code>product["brand"]</code>
IndexError	<code>items[99]</code>
FileNotFoundError	<code>open("missing.csv")</code>
ZeroDivisionError	<code>10 / 0</code>
AttributeError	<code>"abc".push()</code>
PermissionError	เขียนไฟล์ที่เปิดอยู่ใน Excel

```
try:
    price = float(input("ราคา: "))
    total = price * qty

except ValueError as e:
    print(f"กรุณาใส่ตัวเลขเท่านั้น ({e})")

except (TypeError, KeyError) as e:
    print(f"ข้อมูลไม่ถูกต้อง: {e}")

else:
    # ทำงานเมื่อไม่เกิดข้อผิดพลาดเลย
    print(f"ยอดรวม {total:,.2f} บาท")

finally:
    # ทำงานเสมอ ใช้ปิดไฟล์/ปิดการเชื่อมต่อ
    print("จบการคำนวณ")
```

Syntax Error เกิดตอน Python อ่านโค้ด โปรแกรมไม่เริ่มทำงานเลย แก่

Exception เกิดตอนโปรแกรมทำงานไปแล้วส่วนหนึ่ง ต้องเขียนโค้ดดักไว้

หลักการเขียน except ที่ถูกต้อง

```
# แย่ที่สุด: นึกหายเข้ากลีบเมฆ
try:
    process_data()
except:
    pass
```

```
# ยังไม่ดี: กว้างเกินไป ไม่รู้ว่าเกิดอะไร
try:
    process_data()
except Exception:
    print("เกิดข้อผิดพลาด")
```

```
# ดี: จับเฉพาะที่คาดไว้ และบอกรายละเอียด
try:
    with open(path, encoding="utf-8") as f:
        data = json.load(f)

except FileNotFoundError:
    print(f"ไม่พบไฟล์ {path} เริ่มด้วยข้อมูลว่าง")
    data = []

except json.JSONDecodeError as e:
    print(f"ไฟล์เสียหายบรรทัด {e.lineno}: {e.msg}")
    data = []
```

1. จับให้แคบที่สุดเท่าที่รู้

ระบุชนิด Exception ที่คาดว่าจะเกิด

2. อย่าใช้ except: pass

เพราะจะกลืนแม้กระทั่ง KeyboardInterrupt
ตอนกด Ctrl+C

3. จับแล้วต้องทำอะไรสักอย่าง

แจ้งผู้ใช้ บันทึก log หรือใช้ค่าสำรอง

การสร้าง Exception ของตนเอง

```
class PyShopError(Exception):
    """คลาสแม่ของข้อผิดพลาดทั้งหมดในระบบ"""

class ProductNotFoundError(PyShopError):
    def __init__(self, code: str) -> None:
        self.code = code
        super().__init__(f"ไม่พบสินค้ารหัส {code}")

class InsufficientStockError(PyShopError):
    def __init__(self, name, requested, available):
        self.available = available
        super().__init__(
            f"{name}: ต้องการ {requested} ชิ้น "
            f"แต่คงเหลือ {available} ชิ้น")
```

```
try:
    income = sell_product(inv, "PRD-999", 5)
except ProductNotFoundError as e:
    print(f"{e} (รหัส: {e.code})")
except InsufficientStockError as e:
    print(f"ขายได้สูงสุด {e.available} ชิ้น")
except PyShopError as e:
    print(f"ข้อผิดพลาดของระบบ: {e}")
```

```
raise ProductNotFoundError(text) from e
# from e เก็บ traceback เดิมไว้

except PyShopError:
    write_log("เกิดข้อผิดพลาด")
    raise      # ส่งต่อตัวเดิมขึ้นไป
```

การใช้ ValueError กับทุกกรณีทำให้แยกไม่ออกว่าผิดพลาดเพราะอะไร
Exception ของตนเองทำให้แยกแยะสาเหตุได้แม่นยำและแนบข้อมูลเพิ่มเติมไป
ด้วยได้

การใช้ logging แทน print

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)-8s"
        " | %(name)s | %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
    handlers=[
        logging.FileHandler(LOG_PATH,
                             encoding="utf-8"),
        logging.StreamHandler(),
    ],
)

logger = logging.getLogger(__name__)

logger.info("เริ่มระบบ PyShop Mini")
logger.warning("สินค้า %s ใกล้หมด", name)
logger.error("บันทึกไฟล์ไม่สำเร็จ: %s", path)

except Exception:
    logger.exception("เกิดข้อผิดพลาดที่ไม่คาดคิด")
```

ระดับ	ค่า	ใช้เมื่อ
DEBUG	10	รายละเอียดตอนพัฒนา
INFO	20	เหตุการณ์ปกติที่ควรบันทึก
WARNING	30	ผิดปกติแต่ยังไปต่อได้
ERROR	40	ทำงานส่วนนั้นไม่สำเร็จ
CRITICAL	50	ระบบทำงานต่อไม่ได้

logging มีระดับความสำคัญ มีเวลาให้อัตโนมัติ บันทึกลงไฟล์ได้ ปิดตอน Production ได้ด้วยการเปลี่ยนระดับเดียว และบอกได้ว่ามาจากโมดูลไหน

ใช้ `logger.info("ค่า %s", value)` แทน f-string เพราะจะไม่เสียเวลาแปลงข้อความหากระดับ log นั้นถูกปิดอยู่

Debug ด้วย VS Code และการอ่าน Traceback

ปุ่ม	คีย์ลัด	ทำอะไร
Continue	F5	วิ่งต่อจนถึง Breakpoint ถัดไป
Step Over	F10	ทำบรรทัดนี้จบแล้วไปบรรทัดถัดไป
Step Into	F11	เข้าไปดูข้างในฟังก์ชันที่ถูกเรียก
Step Out	Shift+F11	ออกจากฟังก์ชันกลับไปให้ผู้เรียก
Stop	Shift+F5	หยุดการดีบั๊ก

Conditional Breakpoint คลิกขวาที่จุดแดง เลือก Edit Breakpoint แล้วใส่เงื่อนไข เช่น `product.stock < 0` โปรแกรมจะหยุดเฉพาะตอนที่เงื่อนไขเป็นจริง

```
Traceback (most recent call last):
  File "main.py", line 45, in <module>
    main()
  File "main.py", line 38, in main
    income = sell_product(inv, "PRD-001", 100)
  File "src\pyshop\service.py", line 22
    return product.sell(qty)
  File "src\pyshop\product.py", line 58, in sell
    raise ValueError(...)
ValueError: เมาส์ไร้สาย คงเหลือเพียง 25 ชิ้น
```

- 1 อ่านบรรทัดล่างสุดก่อน เพราะบอกชนิดและข้อความของ Error
- 2 ไล่ขึ้นจากล่างไปบน เพื่อดูว่าโค้ดของเราบรรทัดไหนเป็นต้นเหตุ
- 3 บรรทัดคือจุดเริ่มต้นของสายการเรียก
- 4 ถ้าบรรทัดล่างเป็นไฟล์ในไลบรารีภายนอก ให้ไล่ขึ้นมาหาไฟล์ในโปรเจกต์เราที่อยู่ใกล้ที่สุด

WORKSHOP 9.1

ไล่หาสาเหตุของ Bug ด้วย Debugger

ฝึกใช้ Debugger กับโปรแกรมคำนวณส่วนลดที่มีบักซ่อนอยู่ 3 จุด แทนการเดาด้วย print()

จุด	อาการ	สาเหตุ	วิธีแก้
1	TypeError: unsupported operand for *: 'int' and 'NoneType'	discount_rate() ไม่มี return สำหรับกรณี qty น้อยกว่า 3 จึงคืน None	เพิ่ม return 0.0 เป็นบรรทัดสุดท้าย
2	TypeError: can't multiply sequence by non-int of type 'float'	price ของ PRD-003 เป็น str การคูณ str ด้วย int แปลงชนิดตอนอ่านข้อมูล float(order["price"]) คือการทำซ้ำข้อความ	
3	ค่าเฉลี่ยผิด ตัวหารไม่ตรงกับตัวตั้ง	total_amount รวมทุกแถวรวมแถวที่ qty = 0 แต่ total_qty คำนวณเฉพาะแถวที่ qty > 0	ใช้เงื่อนไขกรองเดียวกันทั้งสองบรรทัด และกัน ZeroDivisionError

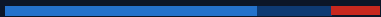
บทเรียนจาก Workshop นี้ ถ้าเราใช้คลาส Product จาก Module 8 บักที่ 2 จะไม่มีทางเกิดเลย เพราะ __post_init__ แปลงชนิดและตรวจสอบให้ตั้งแต่ตอนสร้าง object

10

MODULE 10 · 13:45-14:45

การเขียน Python ติดต่อฐานข้อมูล

เชื่อม Python เข้ากับ PostgreSQL อย่างถูกวิธีและปลอดภัย



ทำไมต้องใช้ฐานข้อมูล

ประเด็น	ไฟล์ CSV / JSON	ฐานข้อมูล (PostgreSQL)
ปริมาณข้อมูล	เหมาะกับหลักพันแถว	หลักล้านแถวขึ้นไป
ค้นหาเฉพาะแถว	ต้องอ่านทั้งไฟล์	ใช้ Index ค้นหาตรงจุด เร็วมาก
หลายคนใช้พร้อมกัน	ทับกันแน่นอน	จัดการให้
ความถูกต้องของข้อมูล	ต้องตรวจสอบทั้งหมด	มี Constraint บังคับให้

หลักสูตรนี้เลือก **PostgreSQL** เพราะทำตามมาตรฐาน SQL ได้ครบถ้วนที่สุด รองรับ JSON ในตัว และเป็นที่นิยมสูงมากในงานสมัยใหม่ ส่วนผู้ที่ใช้ **MySQL** อยู่แล้ว แนวคิดทุกอย่างใช้ได้เหมือนกัน เปลี่ยนเพียงไลบรารีเป็น mysql-connector-python และตัวยัดตำแหน่งค่ายังคงเป็น %s เช่นเดียวกัน

ติดตั้ง PostgreSQL และสร้างตาราง

```
psql -U postgres

CREATE DATABASE pyshop_db;
CREATE USER pyshop_user
  WITH PASSWORD 'รหัสผ่านของคุณ';
GRANT ALL PRIVILEGES ON DATABASE
  pyshop_db TO pyshop_user;

\c pyshop_db
GRANT ALL ON SCHEMA public TO pyshop_user;
\q
```

ค่าที่ต้องจำตอนติดตั้ง Port 5432 · Username postgres ·
Password ที่ตัวเอง จดไว้ให้ดี ลืมแล้วรีเซ็ตยุ่งยาก ตัวติดตั้งจะให้
pgAdmin 4 มาด้วย ใช้จัดการฐานข้อมูลผ่านหน้าเว็บได้สะดวก

```
CREATE TABLE IF NOT EXISTS products (
  id          SERIAL PRIMARY KEY,
  code        VARCHAR(20)  NOT NULL UNIQUE,
  name        VARCHAR(200) NOT NULL,
  price       NUMERIC(12, 2) NOT NULL
              CHECK (price >= 0),
  stock       INTEGER NOT NULL DEFAULT 0
              CHECK (stock >= 0),
  category    VARCHAR(100) NOT NULL
              DEFAULT 'ทั่วไป',
  created_at  TIMESTAMP NOT NULL
              DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_products_category
  ON products(category);
```

ทำไมเงินต้องใช้ NUMERIC ไม่ใช่ FLOAT เพราะ float มีความคลาดเคลื่อนในการเก็บทศนิยม ซึ่งยอมรับไม่ได้ในงานการเงิน ส่วน CHECK ทำให้ฐานข้อมูลบังคับกฎธุรกิจให้ แม้โปรแกรมเผลอส่งค่าติดลบก็ใส่ไม่ได้

ติดตั้ง Driver และเก็บข้อมูลการเชื่อมต่ออย่างปลอดภัย

```
uv add "psycopg[binary]"
uv add python-dotenv
```

ฐานข้อมูล	คำสั่งติดตั้ง
PostgreSQL	<code>uv add "psycopg[binary]"</code>
MySQL / MariaDB	<code>uv add mysql-connector-python</code>
SQLite	มากับ Python (import sqlite3)

```
# .env (เพิ่ม .env ลงใน .gitignore ทันที)
DB_HOST=localhost
DB_PORT=5432
DB_NAME=pyshop_db
DB_USER=pyshop_user
DB_PASSWORD=รหัสผ่านของคุณ
```

```
from dotenv import load_dotenv
load_dotenv()

DB_CONFIG = {
    "host": os.getenv("DB_HOST", "localhost"),
    "port": int(os.getenv("DB_PORT", "5432")),
    "dbname": os.getenv("DB_NAME"),
    "user": os.getenv("DB_USER"),
    "password": os.getenv("DB_PASSWORD", ""),
}
```

ห้ามเขียนรหัสผ่านลงในโค้ดเด็ดขาด เพราะโค้ดจะถูกอัปขึ้น Git แล้วรหัสผ่านหลุด ให้เก็บไว้ในไฟล์ .env ที่ไม่ขึ้น Git แล้วส่ง .env.example ให้เพื่อนรู้

การเชื่อมต่อและส่งงาน SQL

```
import psycopg
from psycopg.rows import dict_row

with psycopg.connect(**DB_CONFIG,
                    row_factory=dict_row) as conn:
    with conn.cursor() as cur:
        cur.execute(
            "SELECT code, name, price "
            "FROM products ORDER BY code")
        rows = cur.fetchall()

for row in rows:
    print(f"{row['code']} {row['name']}")
```

```
Connection (การเชื่อมต่อ)
├─ Cursor (ตัวชี้)
│   └─ execute()
│       └─ fetchall()
└─ commit() / rollback()
```

เมธอด	คืนอะไร	ใช้เมื่อ
<code>fetchone()</code>	1 แถว หรือ None	คาดหวังได้ผลแถวเดียว
<code>fetchall()</code>	list ของทุกแถว	ผลลัพธ์ไม่มากเกินไป
<code>fetchmany(n)</code>	list จำนวน n แถว	แบ่งดึงทีละชุด
<code>rowcount</code>	จำนวนแถวที่กระทบ	ตรวจว่า UPDATE โดนกี่แถว

ปกติ psycopg คืนผลเป็น tuple ต้องอ้างด้วยตำแหน่ง `row[0]` ซึ่งอ่านยากและพังง่ายเมื่อลำดับคอลัมน์เปลี่ยน การใช้ `dict_row` ทำให้อ้างด้วยชื่อคอลัมน์ได้

Parameterized Query — ป้องกัน SQL Injection

```
# อันตรายที่สุด: ต่อข้อความ SQL เอง
code = input("รหัสสินค้า: ")
cur.execute(
    f"SELECT * FROM products "
    f"WHERE code = '{code}'")
```

```
# ผู้ใช้พิมพ์: ' OR '1'='1
# → ดึงข้อมูลทั้งตารางออกมา
```

```
# ผู้ใช้พิมพ์: '; DROP TABLE products; --
# → ตารางหายทั้งตาราง
```

```
# ถูกต้อง: ใช้ %s แล้วส่งค่าเป็น tuple แยก
cur.execute(
    "SELECT * FROM products WHERE code = %s",
    (code,))
#          ต้องเป็น tuple มีคอมมา

cur.execute(
    "SELECT * FROM products WHERE name ILIKE %s",
    (f"%{keyword}%",))
# ใส่ % รอบคำค้นในฝั่ง Python ไม่ใช่ใน SQL
```

กลไกที่ทำให้ปลอดภัย

SQL ถูกส่งไปก่อนแยกต่างหาก ค่าถูกส่งตามทีหลัง จึงถูกมองเป็น "ข้อมูล" เท่านั้น ไม่ใช่คำสั่ง

ข้อควรระวังที่พบบ่อย

(code) ไม่ใช่ tuple แต่เป็นวงเล็บธรรมดา ต้องเขียน (code,) มีคอมมา ปิดท้ายเสมอเมื่อมีค่าเดียว

Transaction — ทำสำเร็จทั้งหมด หรือไม่ทำเลย

```

conn = psycopgg.connect(**DB_CONFIG)
try:
    with conn.cursor() as cur:
        cur.execute(
            "UPDATE products SET stock = stock - %s "
            "WHERE code = %s AND stock >= %s",
            (qty, code, qty))
        if cur.rowcount == 0:
            raise ValueError("คงเหลือไม่พอ")

        cur.execute(
            "INSERT INTO sales (product_code, qty, "
            "amount) VALUES (%s, %s, %s)",
            (code, qty, amount))

    conn.commit()      # ยืนยันทั้งสองคำสั่งพร้อมกัน

except Exception as e:
    conn.rollback()    # ย้อนกลับทั้งหมด

finally:
    conn.close()

```

ตัวอย่างที่ต้องใช้ Transaction การขายสินค้าคือลดสต็อกและบันทึกใบเสร็จ ถ้าลดสต็อกสำเร็จแต่บันทึกใบเสร็จล้มเหลว ข้อมูลจะไม่ตรงกันทันที

psycopg 3 เมื่อใช้ `with psycopgg.connect(...)` จะ `commit` ให้อัตโนมัติเมื่อออกจากบล็อกโดยไม่มีข้อผิดพลาด และ `rollback` ให้อัตโนมัติเมื่อเกิดข้อผิดพลาด

แต่ในกรณีที่ต้องควบคุมละเอียด เช่น ทำหลายขั้นตอนแล้วตัดสินใจกลางทาง ให้เขียน `commit()` และ `rollback()` เองอย่างชัดเจน

เพิ่มข้อมูลหลายแถวและ Upsert

```
rows = [
    ("PRD-001", "เมาส์ไร้สาย", 890, 25, "อุปกรณ์เสริม"),
    ("PRD-002", "คีย์บอร์ด", 1290, 8, "อุปกรณ์เสริม"),
]

with conn.cursor() as cur:
    cur.executemany("""
        INSERT INTO products
            (code, name, price, stock, category)
        VALUES (%s, %s, %s, %s, %s)
        ON CONFLICT (code) DO UPDATE
        SET name = EXCLUDED.name,
            price = EXCLUDED.price,
            stock = EXCLUDED.stock,
            updated_at = CURRENT_TIMESTAMP
    """, rows)
```

```
# รับค่า id ที่ฐานข้อมูลสร้างให้กลับมา
cur.execute(
    "INSERT INTO products (code, name, price) "
    "VALUES (%s, %s, %s) RETURNING id",
    ("PRD-006", "ที่วางแล็ปท็อป", 690))
new_id = cur.fetchone()["id"]
```

executemany เพิ่มหลายแถวในครั้งเดียว เร็วกว่าวน **execute** ทีละแถวมาก

ON CONFLICT ... DO UPDATE แปลว่า "ถ้ารหัสนี้มีอยู่แล้วให้อัปเดตแทนการเพิ่มใหม่" ทำให้เขียนสคริปต์นำเข้าข้อมูลซ้ำได้โดยไม่เกิดข้อผิดพลาด เรียกว่า Upsert

WORKSHOP 10.1

เชื่อม PyShop Mini เข้ากับ PostgreSQL

สร้างชั้นเข้าถึงข้อมูล (Repository) ที่ทำ CRUD ครบทั้ง 4 คำสั่ง โดยใช้ Parameterized Query ทั้งหมด

```
# src/pyshop/database.py
@contextmanager
def get_connection()
def init_db()

# CREATE
def insert_product(product) -> int
def upsert_many(products) -> int
# READ
def fetch_all() -> list[Product]
def fetch_by_code(code) -> Product | None
def search_products(keyword) -> list
def summary_by_category() -> list[dict]
# UPDATE
def update_stock(code, change) -> bool
def update_price(code, price) -> bool
# DELETE
def delete_product(code) -> bool
```

นำเข้า 5 รายการ และ **รันซ้ำอีกครั้งต้องไม่เกิด error** เพราะใช้ ON CONFLICT

update_stock('PRD-003', -3) ต้องคืน **False** เพราะสต็อกเป็น 0 อยู่แล้ว ซึ่งพิสูจน์ว่ากฎกันสต็อกติดลบทำงาน

การค้นด้วย ' OR '1'='1' ต้องได้ **0 รายการ** ไม่ใช่ทั้งตาราง ซึ่งพิสูจน์ว่า Parameterized Query ป้องกันได้จริง

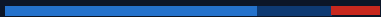
uv run workshop10.py

11

MODULE 11 · 14:45-15:15

Git Version Control

เก็บประวัติทุกการเปลี่ยนแปลง ย้อนกลับได้ทุกจุด และให้หลายคนแก้ไขไฟล์เดียวกันได้โดยไม่ทับกัน



แนวคิด 3 พื้นที่ และคำสั่ง Git พื้นฐาน

Working Directory (ไฟล์ที่คุณแก้) —git add→ Staging Area (เตรียมจะบันทึก) —git commit→ Repository (ประวัติถาวร) —git push→ GitHub

```
git config --global user.name "Samit Koyom"
git config --global init.defaultBranch main
git config --global core.autocrlf true
```

```
git status          # ใ้บ่ย่อที่สุด
git add .
git commit -m "เพิ่มคลาส Product"
```

```
git log --oneline
git log --oneline --graph --all
git diff / git diff --staged
```

```
git restore main.py    # ทิ้งการแก้ไข
git revert <commit-id> # ย้อนอย่างปลอดภัย
```

การเขียนข้อความ commit ที่ดี

ไม่ดี	ดี
update	เพิ่มฟังก์ชันค้นหาสินค้าจากรหัสและชื่อ
fix bug	แก้บั๊ก discount_rate คีน None เมื่อ qty น้อยกว่า 3
แก้ไขไฟล์	เปลี่ยนจาก dict เป็นคลาส Product เพื่อตรวจสอบข้อมูลอัตโนมัติ

git restore <ไฟล์> ลบการแก้ไขที่ยังไม่ commit ทั้งถาวร และไม่มีทางกู้คืนได้ เพราะ Git ไม่เคยเห็นการแก้ไขนั้นเลย

.gitignore และการทำงานกับ Branch

```
# Virtual Environment
.venv/

# Python cache
__pycache__/
*.py[cod]
.ruff_cache/

# ความลับ - ห้ามขึ้น Git เด็ดขาด
.env
*.key
credentials.json

# ข้อมูลและไฟล์ผลลัพธ์
data/*.bak.csv
data/*.log

# Editor
.vscode/*
!.vscode/settings.json
```

```
git branch
git switch -c feature/gui-form

git add . && git commit -m "เพิ่มหน้าจอฟอร์ม"

git switch main
git merge feature/gui-form
git branch -d feature/gui-form
```



สิ่งที่ห้ามขึ้น Git โดยเด็ดขาด ไฟล์ .env รหัสผ่าน API Key ไฟล์ Certificate และข้อมูลส่วนบุคคล เพราะ Git จำกุกอย่างตลอดไป ต่อให้ลบไฟล์แล้ว commit ใหม่ ประวัติเดิมก็ยังมีข้อมูลนั้นอยู่

นำ PyShop Mini ขึ้น GitHub

```
# ส่งขึ้น GitHub ครั้งแรก
git status
git add .
git commit -m "PyShop Mini: ระบบจัดการสินค้า"

git remote add origin \
  https://github.com/your-name/pyshop.git
git branch -M main
git push -u origin main

# ใช้งานประจำวัน
git push
git pull
```

```
# รับโปรเจกต์จากภายนอกมาใช้งาน
git clone https://github.com/team/project.git
cd project
uv sync
uv run main.py
```

ผลลัพธ์ที่คาดหวัง

เปิดหน้า repository บน GitHub แล้วเห็นซอร์สโค้ดครบ แต่ต้องไม่เห็นไฟล์เดอร์ `.venv` และไฟล์ `.env` ถ้าเห็นแสดงว่า `.gitignore` ยังไม่ทำงาน ให้แก้แล้วรัน `git rm -r --cached .venv .env` ก่อน commit ใหม่

นี่คือเหตุผลที่ `uv.lock` ต้องขึ้น Git

คนที่โคลนโปรเจกต์ไปจะได้ไลบรารีเวอร์ชันเดียวกันเป๊ะกับเครื่องท่าน ไม่มีปัญหา "เครื่องผมรันไม่ได้" อีกต่อไป

โจทย์ต่อยอด

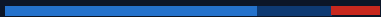
จับคู่กับเพื่อนในห้อง ให้เพื่อน `git clone` โปรเจกต์ของท่านแล้วรัน `uv sync` และ `uv run main.py` ถ้าเพื่อนรันได้โดยไม่ต้องถามอะไรเพิ่ม แสดงว่าโปรเจกต์ของท่านสมบูรณ์แล้ว

12

MODULE 12 · 15:15-15:45

Python GUI

โปรแกรม Console ใช้งานได้ดีสำหรับนักพัฒนา แต่ผู้ใช้ทั่วไปต้องการหน้าต่างที่คลิกได้



ภาพรวมไลบรารีสำหรับสร้าง GUI

ไลบรารี	ข้อดี	ข้อเสีย	เหมาะกับ
Tkinter	มากับ Python ไม่ต้องติดตั้ง เรียนรู้เร็ว	หน้าต่างเก่า	เครื่องมือภายในองค์กร โปรแกรมเล็ก
CustomTkinter	ต่อยอดจาก Tkinter หน้าตากันสมัย มี Dark Mode	ต้องติดตั้งเพิ่ม	โปรแกรมที่ต้องการความสวยงาม
PyQt / PySide	ครบเครื่องที่สุด มีเครื่องมือออกแบบ UI	เรียนรู้ยาก ลิสสิทธ์ต้องตรวจสอบ	แอปเดสก์ท็อประดับมืออาชีพ
Flet	เขียนครั้งเดียวได้ทั้ง Desktop / Web / Mobile	ยังใหม่ ชุมชนเล็กกว่า	โปรเจกต์ที่ต้องการหลายแพลตฟอร์ม

หลักสูตรนี้ใช้ **Tkinter** เป็นฐาน เพราะมากับ Python อยู่แล้ว ทุกคนใช้ได้ทันที แล้วจะแสดงให้เห็นว่า **CustomTkinter** ยกระดับหน้าต่างได้อย่างไร ด้วยการแก้ไขโค้ดเพียงเล็กน้อย

หน้าต่างแรก Widget และการจัดวาง Layout

```
import tkinter as tk

root = tk.Tk()
root.title("PyShop Mini")
root.geometry("400x200")

label = tk.Label(root, text="ยินดีต้อนรับ",
                  font=("Tahoma", 14))
label.pack(pady=20)

tk.Button(root, text="ปิดโปรแกรม",
          command=root.destroy).pack()

root.mainloop()    # ต้องเป็นบรรทัดสุดท้าย
```

mainloop() คือลูปที่คอยรับการคลิกและการพิมพ์ของผู้ใช้ โค้ดที่เขียนต่อจากบรรทัดนี้จะทำงานก็ต่อเมื่อผู้ใช้ปิดหน้าต่างแล้วเท่านั้น

ฟอนต์ภาษาไทยบน Windows ใช้ Tahoma หรือ Leelawadee UI จะแสดง

```
# pack() - เรียงต่อกันตามทิศทาง
widget.pack(side="top", fill="x", padx=10)

# grid() - เหมาะกับฟอร์มมากที่สุด
tk.Label(root, text="ชื่อสินค้า:").grid(
    row=0, column=0, sticky="e", padx=5)
tk.Entry(root).grid(
    row=0, column=1, sticky="ew", padx=5)
root.columnconfigure(1, weight=1)

# place() - พิกัดตายตัว ไม่แนะนำ
widget.place(x=50, y=100, width=200)
```

Widget ที่ใช้บ่อย

Label · Entry · Text · Button · Combobox · Checkbutton ·
Radiobutton · **Treeview** (ตารางข้อมูล) · Frame · messagebox

Tkinter มีระบบจัดวาง 3 แบบ ห้ามใช้ปนกันภายใน Frame เดียวกัน

การรับ-ส่งค่าจากฟอร์มและการผูก Event

```
class ProductForm(tk.Frame):
    def __init__(self, master):
        super().__init__(master, padx=12, pady=12)

        # ตัวแปรพิเศษที่ผูกกับ Widget สองทาง
        self.var_code = tk.StringVar()
        self.var_name = tk.StringVar()
        self.var_price = tk.StringVar(value="0")

        tk.Entry(self,
                 textvariable=self.var_code).grid(...)

        # ผูก Event ด้วย command
        tk.Button(self, text="บันทึก",
                 command=self.on_save).grid(...)

        # ผูกคีย์ Enter ให้เท่ากับกดปุ่มบันทึก
        master.bind("<Return>",
                    lambda event: self.on_save())

    def on_save(self):
        data = self.get_data()
        if not data["code"]:
```

ตัวแปรพิเศษ

ใช้กับ

tk.StringVar()

ข้อความ

tk.IntVar()

จำนวนเต็ม

tk.DoubleVar()

ทศนิยม

tk.BooleanVar()

ค่าจริง/เท็จ (Checkbox)

กล่องข้อความ

ใช้เมื่อ

showinfo()

แจ้งข้อมูล

showwarning()

เตือน

showerror()

แจ้งข้อผิดพลาด

askyesno()

ถามยืนยันก่อนลบ คืน True/False

CustomTkinter — ยกระดับหน้าต่างให้ทันสมัย

```
uv add customtkinter

import customtkinter as ctk

ctk.set_appearance_mode("dark")
ctk.set_default_color_theme("blue")

app = ctk.CTk()
app.title("PyShop Mini")
app.geometry("460x300")

ctk.CTkLabel(app, text="PyShop Mini",
             font=("Tahoma", 22, "bold")).pack(pady=20)

entry = ctk.CTkEntry(app,
                    placeholder_text="ค้นหาสินค้า...",
                    width=300, height=36)
entry.pack(pady=10)

ctk.CTkButton(app, text="ค้นหา").pack(pady=10)
ctk.CTkSwitch(app, text="โหมดกลางคืน").pack()
```

Tkinter	CustomTkinter	ความต่าง
tk.Tk()	ctk.CTk()	รองรับ Dark Mode
tk.Label	ctk.CTkLabel	มุมโค้ง สีสวยกว่า
tk.Entry	ctk.CTkEntry	มี placeholder_text
tk.Button	ctk.CTkButton	มี hover effect
—	ctk.CTkSwitch	สวิตช์เปิด/ปิดสมัยใหม่

CustomTkinter ทำงานบนพื้นฐานของ Tkinter ทั้งหมด ดังนั้นทุก
 อย่างที่เรียนมา (grid, pack, StringVar, command) ใช้ได้เหมือน
 เดิม เปลี่ยนแค่ชื่อคลาสของ Widget เท่านั้น และยังใช้ ttk.Treeview
 ร่วมกับ CustomTkinter ได้

MODULE 13 · 15:45-16:00

สถาปัตยกรรมของโปรแกรมที่เราสร้าง

PRESENTATION — ชั้นแสดงผล

app_gui.py

Tkinter: ฟอรม + ตาราง + ปุ่ม

↓ เรียกใช้

DOMAIN — ชั้นตรรกะธุรกิจ

src/pyshop/product.py

class Product, class Inventory — ตรวจสอบข้อมูลและคำนวณ

↓ เรียกใช้

DATA ACCESS — ชั้นเข้าถึงข้อมูล

src/pyshop/database.py

psycopg → PostgreSQL (CRUD ด้วย Parameterized Query)

แต่ละชั้นเปลี่ยนได้อิสระ เช่น เปลี่ยนจาก PostgreSQL เป็น MySQL แค่ database.py ชั้นอื่นไม่ต้องแตะ

WORKSHOP ปิดท้าย

PyShop Mini v2 — GUI + PostgreSQL

รับข้อมูลจากผู้ใช้งานผ่าน GUI ประมวลผลด้วยคลาส Product บันทึกลง PostgreSQL แสดงผลในตาราง และส่งออกรายงานเป็น CSV

```
class PyShopApp(tk.Tk):  
    _build_header()      # แถบหัวสีเข้ม  
    _build_form()        # ฟอर्मกรอกข้อมูล  
    _build_toolbar()     # 6 ปุ่มเครื่องมือ  
    _build_table()       # Treeview  
    _build_statusbar()   # สรุปลำดับและมูลค่า  
  
    on_add() / on_adjust() / on_delete()  
    on_search() / on_export() / refresh()
```

ตารางระบายสีแดงสำหรับสินค้าหมดและสีเหลืองสำหรับใกล้หมด แถบสถานะด้านล่างสรุปจำนวนกับมูลค่ารวม

ทดสอบให้ครบ 5 ข้อ

- 1 เพิ่มสินค้าใหม่ ตารางอัปเดตทันที และปิดโปรแกรมเปิดใหม่ข้อมูลยังอยู่
- 2 คลิกแถวในตาราง ฟอर्मด้านบนเติมค่าให้อัตโนมัติ
- 3 เลือกจอกภาพที่สต็อก 0 แล้วกดตัดสต็อก ต้องขึ้นเตือนว่าสินค้าไม่พอ
- 4 กดส่งออก CSV ได้ไฟล์ที่เปิดด้วย Excel แล้วภาษาไทยไม่เพี้ยน
- 5 เพิ่มสินค้าด้วยรหัสที่มีอยู่แล้ว ต้องขึ้นข้อความแจ้ง ไม่ใช่โปรแกรมพัง

ข้อสังเกตที่สำคัญที่สุดของหลักสูตรนี้ ไฟล์ app_gui.py ไม่มีคำสั่ง SQL แม้แต่บรรทัดเดียว และไม่มีสูตรคำนวณมูลค่าหรือ VAT เลย มันทำหน้าที่เพียงรับค่าจากผู้ใช้งานแล้วเรียกใช้ Product กับ database เท่านั้น

```
uv run app_gui.py
```

ใช้ AI เป็นผู้ช่วย และเส้นทางการเรียนรู้ต่อ

ใช้อย่างนี้	อย่าใช้อย่างนี้
"อธิบายว่าโค้ดนี้ทำงานอย่างไรทีละบรรทัด"	"เขียนโปรแกรมทั้งหมดให้หน่อย" แล้วส่งงานเลย
"ทำไม error นี้ถึงเกิด และแก้ยังไง"	คัดลอกโค้ดที่ไม่เข้าใจไปวางในงานจริง
"มีวิธีเขียนแบบอื่นที่อ่านง่ายกว่านี้ไหม"	ใช้แทนการอ่านเอกสารทางการทั้งหมด
"ช่วยเขียน docstring และ type hints ให้โค้ดนี้"	เชื่อทุกอย่างโดยไม่ทดสอบ
"ช่วยทบทวนโค้ดนี้ว่ามีช่องโหว่ความปลอดภัยไหม"	ส่งรหัสผ่านหรือข้อมูลลูกค้าเข้าไปใน prompt

Python Basic 2026 (จบแล้ว)

|

└─▶ Web Flask → FastAPI → Django

└─▶ Data Pandas → Plotly → SQL เชิงลึก

└─▶ AI / ML NumPy → scikit-learn → PyTorch

└─▶ Automation openpyxl → Playwright

└─▶ Python Advanced (หลักสูตรต่อเนื่อง)

Decorator, Generator, Async,

pytest, Packaging, Performance

กฎเหล็กข้อเดียวที่ควรจำ

ถ้าท่านอธิบายโค้ดที่ AI เขียนให้เพื่อนฟังไม่ได้ อย่าเพิ่งใช้โค้ดนั้น เพราะเมื่อเกิดปัญหาในระบบจริง คนที่ต้องแก้คือท่าน ไม่ใช่ AI

หลังจบการอบรม

แบบทดสอบหลังเรียน (Post-test)

เข้าทำแบบทดสอบที่

wayground.com/join

กรอก Join Code ที่วิทยากรแจ้งหน้าห้อง เพื่อวัดผลสัมฤทธิ์การเรียนรู้และเปรียบเทียบกับคะแนนก่อนเรียน

บริษัท ไอทีจีเนียส เอ็นจิเนียริ่ง จำกัด

โทร. 02-570-8449 · Line ID: @itgenius · www.itgenius.co.th

หลักสูตรต่อเนื่องที่แนะนำ: Python Advanced



SCAN ME